Kubernetes Networking and Ingress

every pod gets an IP; Services give it a name; policies decide who may talk

Watch the 4-min video

> **In one line:** the CNI gives every pod a routable IP, a Service gives a changing set of pods one stable virtual IP and DNS name, kube-proxy or eBPF on each node translates that IP to a ready pod, an ingress controller or Gateway brings outside traffic in, and NetworkPolicies (default deny + explicit allows) stop the traffic you don't want.

1 Three traffic questions

Inside one cluster: the orders service calls payments **by name** (east-west traffic); a browser reaches the API through **one load balancer** (north-south traffic); and a compromised pod in the dev namespace tries to connect to the production database. Knowing each layer is also how you debug the classic "it can't connect": DNS? Service selector? Pod not Ready? A NetworkPolicy dropping packets?

2 Every pod gets an IP: the CNI

The Kubernetes network model has three rules: every pod gets its own cluster-wide IP; pods can reach all other pods **without NAT**; and node agents can reach all pods on their node. A **CNI plugin** implements them: it allocates an address when a pod starts, wires its virtual interface, and makes sure packets reach the right node. Each node usually owns a slice of the pod range (for example a /24, 256 addresses). Three common designs:

- **Overlay** (VXLAN, e.g. Flannel, Calico in overlay mode, Azure CNI Overlay): pod IPs live in a private range and are tunnelled between nodes. Saves VNet addresses; small encapsulation cost.
- **VPC-native** (AWS VPC CNI, classic Azure CNI, GKE VPC-native): pods get real VNet/VPC addresses, routable without tunnels. Simple and fast, but large clusters can exhaust subnets.
- **eBPF-based** (Cilium): programs the kernel directly for routing, load balancing and policy, and can replace kube-proxy entirely.

3 Services, DNS and kube-proxy

Pods come and go, so their IPs are useless as addresses. A **Service** selects pods by label and gets a stable **ClusterIP** (a virtual IP that no interface owns) and a DNS name `<service>.<namespace>.svc.cluster.local`. The control plane keeps **EndpointSlices** listing the Service's pod IPs, including only pods that pass their **readiness probe**. On every node, **kube-proxy** (iptables, IPVS or nftables mode) or Cilium's eBPF programs rewrite connections to the virtual IP into connections to one of those pods.

Service type	What it gives you	Typical use
ClusterIP	internal virtual IP + DNS name	service-to-service calls
Headless (`clusterIP: None`)	DNS returns pod IPs directly	StatefulSets, client-side balancing (lesson 59)
NodePort	a port (30000–32767) open on every node	building block for external load balancers
LoadBalancer	a cloud load balancer with an external or internal IP	exposing one service, or the ingress controller itself
ExternalName	a DNS CNAME to an outside name	pointing at a managed database by a cluster name

4 Service-to-service, step by step

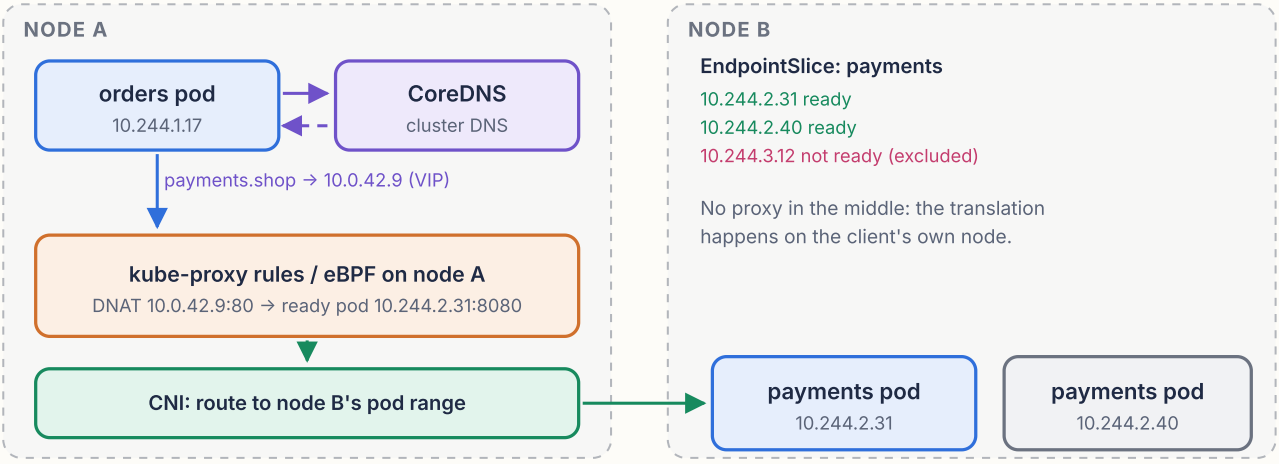


Figure 1. orders → payments. DNS gives a virtual IP; node A's rules pick a ready pod and rewrite the destination; the CNI carries the packet to node B.

1. orders resolves `payments`. The pod's `/etc/resolv.conf` search list expands it to `payments.shop.svc.cluster.local` (from another namespace you'd use `payments.shop`).
2. CoreDNS answers with the Service's ClusterIP, e.g. 10.0.42.9.
3. orders opens a TCP connection to 10.0.42.9:80. Before it leaves node A, the kernel rules DNAT it to a ready pod, 10.244.2.31:8080, chosen at random (iptables) or by a scheduler such as round robin (IPVS).
4. The CNI routes the packet to node B; the reply is un-NATed on the way back. There is no central proxy hop.

Two consequences worth knowing. Balancing is **per connection**, not per request: a gRPC or HTTP/2 client that keeps one long-lived connection sends **all** its requests to one pod, so use client-side balancing over a headless Service or a service mesh. And the default `ndots:5` setting makes a lookup of an external name like `api.stripe.com` try the cluster search domains first; add a trailing dot or tune ndots if DNS load becomes a problem.

5 Traffic from outside: ingress and Gateway API

Giving every service its own cloud load balancer is expensive and scatters TLS. Instead, one `LoadBalancer` Service fronts an **ingress controller**: a proxy (NGINX, Traefik, HAProxy, Envoy-based) running as pods. **Ingress** resources tell it to route by host and path to Services, and to terminate TLS.

Cloud controllers (AWS Load Balancer Controller, Azure Application Gateway for Containers) instead program the cloud's L7 load balancer directly.



Figure 2. North-south: one public load balancer, one ingress tier, then host and path rules to ordinary ClusterIP Services.

The Ingress API is frozen: it covers host/path routing and TLS, and everything else lives in controller-specific annotations. The **Gateway API** (GA since 2023) replaces it with role-oriented, portable objects and built-in features such as header matching, **weighted traffic splitting** for canaries, redirects, and TCP/gRPC routes. Most controllers now support it, and new clusters should start there. (The community ingress-nginx project announced its retirement in late 2025, with maintenance ending in March 2026, which pushed many teams to migrate.)

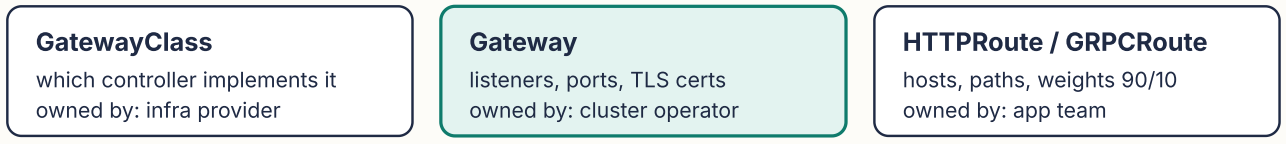


Figure 3. Gateway API splits responsibilities so app teams can change routes without touching shared load balancer settings.

6 NetworkPolicies: firewalls for pods

By default every pod can reach every pod, in every namespace: convenient and dangerous. A **NetworkPolicy** selects pods and lists allowed sources (ingress) or destinations (egress) by pod labels, namespace labels, IP blocks and ports. Rules to remember:

- Once **any** policy selects a pod for a direction, everything not explicitly allowed in that direction is dropped. Policies are **additive**; there is no "deny" rule.
- **Default deny** per namespace: a policy with `podSelector: {}` and `policyTypes: [Ingress, Egress]` and no rules. Then allow only needed flows. If you deny egress, remember to allow DNS (port 53 to kube-dns), or everything "breaks".
- Policies are **enforced by the CNI**. A plugin without policy support (basic Flannel) silently ignores them; Calico, Cilium and Azure's network policy engine enforce them.
- In a `from` list, `namespaceSelector` and `podSelector` in the **same item** mean AND; as **separate items** (each with its own dash) they mean OR. One misplaced dash opens the database to every pod in the cluster with the label.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: postgres-from-orders, namespace: data }
spec:
  podSelector: { matchLabels: { app: postgres } }
  policyTypes: [Ingress]
  ingress:
  - from:
    - namespaceSelector:                       # same list item:
        matchLabels: { kubernetes.io/metadata.name: shop }
      podSelector:                             # ns AND label
        matchLabels: { app: orders }
    ports: [ { protocol: TCP, port: 5432 } ]
```

Figure 4. The worked example's policy. Both selectors sit under one list item, so a source must be in namespace `shop` AND labelled `app=orders`.

WORKED EXAMPLE: LOCKING DOWN THE DATABASE

The `data` namespace first gets a default-deny ingress policy. Then the policy above allows TCP 5432 to `app=postgres` pods only from pods labelled `app=orders` in namespace `shop`. The namespace is matched by the label `kubernetes.io/metadata.name`, which Kubernetes sets automatically, so nobody can gain access just by adding a label to their own namespace. When the compromised pod in `dev` tries to connect, the CNI drops its SYN packets and the connection times out. orders keeps working. To check the setup, run a test pod in each namespace and try `nc -zv postgres.data 5432`: it should succeed only from orders.

COMMON MISTAKES

- Service selector labels that don't match the pods: the Service exists but has no endpoints.
- A pod failing readiness, so it silently receives no traffic.
- Default-deny egress without a DNS exception.
- Writing NetworkPolicies on a CNI that doesn't enforce them, and assuming you're protected.

7 Key terms

CNI: plugin that gives pods IPs and routes between nodes

ClusterIP: stable virtual IP for a Service

EndpointSlice: the list of ready pod IPs behind a Service

kube-proxy: per-node agent programming Service translation rules

Ingress / Gateway API: objects routing external HTTP(S) traffic to Services

NetworkPolicy: label-based allow-list firewall for pods

8 Interview questions

Q1. How does service-to-service traffic work in Kubernetes? BASIC

The pod resolves the Service's DNS name to its ClusterIP; kube-proxy or eBPF rules on the client's node rewrite the connection to one ready pod; the CNI routes pod-to-pod traffic across nodes.

Q2. How do you restrict it? BASIC

With NetworkPolicies, enforced by the CNI: start from default deny per namespace and allow only required flows by namespace, pod labels and ports.

Q3. What's the difference between Ingress and a LoadBalancer Service? BASIC

A LoadBalancer Service gives one Service an L4 cloud load balancer; an ingress controller is one shared L7 entry point that routes many hosts and paths to many Services and terminates TLS.

Q4. A Service returns connection refused, but the pods are running. What do you check? SCENARIO

Whether the Service has endpoints (selector labels match, pods Ready), whether targetPort matches the container port, and whether a NetworkPolicy blocks the source.

Q5. Your gRPC service has 10 pods but one gets almost all traffic. Why? SCENARIO

kube-proxy balances per connection and gRPC reuses one long-lived HTTP/2 connection; use client-side balancing with a headless Service or a mesh / L7 proxy that balances per request.

Q6. You added a NetworkPolicy allowing only port 5432 egress, and now the app can't reach the database by name. Why? TRICKY

Egress is now default-deny for those pods, including DNS; add an egress rule to kube-dns on port 53 UDP and TCP.

Q7. What's the difference between two selectors in one 'from' item and in two items? TRICKY

One item ANDs them (that namespace and that label); separate items OR them, which is usually far broader than intended.

Next: SD-V-011 · Infrastructure as Code backendroadmap.com